

ROBOTIC MANIPULATION

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2024

Last modified 2025-8-26.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Fall 2024 semester.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

APPENDIX D

Setting up your own "Manipulation Station"

Simulation is an extremely powerful tool, and the fact that we can give it away freely makes it a powerful equalizer. Just a few years ago, nobody believed that you could develop a manipulation system in simulation and expect it to work in reality. Nowadays, though, we are seeing consistent evidence that our simulation fidelity is high enough that it can achieve *robust* performance in simulation, then we are optimistic that it can transfer to reality.

1

Of course, there is no substitute to watching your ideas/code bring life to a real physical robot. The goal of this chapter is to help commoditize the robot hardware infrastructure, to the extent possible, by providing the bill of materials, the low-level drivers, and basic setup instructions to get you up and running with your very own manipulation station.

D.1 MESSAGE PASSING

DRAKE supports multiple message passing subsystems (including ROS1), and for the most part I have no strong opinions about what you should use. We use **LCM** for this class because (1) it is lightweight as a dependency (it would be very painful/restrictive to have ROS as a mandatory dependency for **DRAKE**) and (2) we prefer multicast UDP to tcp/ip for the lowest level communications to/from the robot.

As a result, the primary drivers we list/provide below are based on LCM. I'll try to point to ROS equivalents when possible. If you already have an established ROS workflow and still want to use our drivers, then it is also possible to use simple lcm2ros/ros2lcm "bridge" executables to e.g. publish the LCM messages on a ROS topic (for logging, etc).

D.2 KUKA LBR IIWA + SCHUNK WSG GRIPPER

We have made extensive use of both the iiwa7 R800 and the iiwa14 R820; **DRAKE** provides URDFs for both models and the drivers should work for either. We use the iiwa7 for the labs in this course because it is slightly smaller and we do not need the end-effector payload.

- [IIWA LCM driver](#)

We've been quite happy with the Schunk WSG 50 gripper as a reliable and accurate two-fingered gripper solution.

- [Schunk LCM driver](#)

Note: We are evaluating the Franka Panda as an alternative platform (and are likely to provide drivers for it here soon).

D.3 FRANKA PANDA

The Franka Panda series of robots are a very popular iiwa alternative. We use a driver that communicates with it directly over LCM.

- [Franka Panda driver](#)

[Using the ROS drivers directly](#) is certainly possible, too.

D.4 INTEL REALSENSE D415 DEPTH CAMERAS

The D415 has been the camera of choice because of its relatively good minimum range and due to the fact that multiple cameras do not interfere with each other when viewing the same scene.

- [LCM Driver](#)
- [Camera calibration toolbox](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

